

4. Generator

ES6에는 중간에 중지한 다음 중지된 위치에서 계속할 수 있는 기능으로 별표 * 문자를 함수 이름 앞에 붙이고 하나 이상의 yield 문을 포함합니다. **yield** 키워드는 이터레이터 객체를 반환합니다.

```
function * generator_name() {  
  yield value1  
  ...  
  yield valueN  
}
```

```
// 기명 함수  
const sum = function sum (num1, num2) {  
  console.log("sum : pending" );  
  return new Promise(function(resolve, reject) {  
    setTimeout(function() {  
      let result = num1 + num2;  
      result = `${num1} + ${num2} = ${result}`;  
      resolve(result);  
    }, 2000);  
  });  
}  
  
// 익명 함수  
const minus = function (num1, num2) {  
  console.log("minus : pending" );  
  return new Promise(function(resolve, reject) {  
    setTimeout(function() {  
      let result = num1 - num2;  
      result = `${num1} - ${num2} = ${result}`;  
      resolve(result);  
    }, 1500);  
  });  
}  
  
// 화살표 함수 적용  
const multiply = (num1, num2) => {  
  console.log("multiply : pending" );  
  return new Promise((resolve, reject) => {  
    setTimeout( () => {  
      let result = num1 * num2;  
      result = `${num1} * ${num2} = ${result}`;  
      resolve(result);  
    }, 1000);  
  });  
}  
  
// 화살표 함수 적용
```

```
const divide = (num1, num2) => {
  console.log("divide : pending" );
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let result = num1 / num2;
      result = `${num1} / ${num2} = ${result}`;
      resolve(result);
    }, 1000);
  });
}
```

```
// 제너레이터 함수 정의
async function * fnGenerator(){
  yield await sum(100, 200);
  yield await minus(100, 200);
  yield await multiply(100, 200);
  yield await divide (100, 200);
}

let str = '';

async function generate() {
  for await (const val of fnGenerator()) {
    str = str + "\n" + val;
  }
  console.log(str);
}

async function generate01() {
  let fnItems = fnGenerator();
  const sum = await fnItems.next();
  const minus = await fnItems.next();
  const multiply = await fnItems.next();
  const divide = await fnItems.next();

  console.log(sum.value);
  console.log(minus.value);
  console.log(multiply.value);
  console.log(divide.value);
}

generate();
generate01();
```

```
sum : pending
sum : pending
: ▶ Promise {<pending>}
minus : pending
minus : pending
multiply : pending
multiply : pending
divide : pending
divide : pending

100 + 200 = 300
100 - 200 = -100
100 * 200 = 20000
100 / 200 = 0.5

100 + 200 = 300
100 - 200 = -100
100 * 200 = 20000
100 / 200 = 0.5
```

필요한 학습 : <https://davidwalsh.name/async-generators>